Лекция 2. Работа с классами в Python

Погружаемся в детали: от чертежа до работающей программы

В прошлой лекции мы говорили об объектно-ориентированной парадигме в целом. Теперь пришло время надеть каску и взять в руки инструменты — мы начинаем строить классы. Мы разберём каждый кирпичик, из которого состоит класс, и посмотрим, как Python обрабатывает наш код «под капотом».

1. Зачем нам классы? (Мотивация на примере)

Представьте, что мы пишем программу для учёта книг в библиотеке. Без ООП мы бы использовали словари:

```python
book1 = {"title": "Маленький принц", "author": "Антуан де Сент-Экзюпери", "pages": 96, "in_stock": True}
book2 = {"title": "Приключения Тома Сойера", "author": "Марк Твен", "pages": 240, "in_stock": False}

def display_book(book):
    status = "в наличии" if book["in_stock"] else "нет в наличии"
    print(f"{book['title']} — {status}")

def borrow_book(book):
    if book["in_stock"]:
        book["in_stock"] = False
        print(f"Книга '{book['title']}' выдана")
    else:
        print("Книги нет в наличии")
```

Вроде бы работает. Но представьте, что у нас 1000 книг и 50 функций, которые их обрабатывают. Возникают проблемы:

1. **Нет единого места, где описано, какие поля есть у книги.** Кто-то может создать книгу с ключом `"author"`, а кто-то — с `"writer"`. Функции сломаются.
2. **Функции разбросаны по файлам.** Трудно понять, какие операции вообще возможны с книгой.
3. **Нет контроля.** Любой может написать `book1["in_stock"] = "maybe"`, и наша логика проверки сломается.

Классы решают эти проблемы:

- Они группируют данные и операции над ними в одном месте.
- Они задают чёткую структуру (чертёж) для всех объектов.
- Они позволяют контролировать доступ к данным.

2. Самый простой класс и его объект

2.1. Создаём класс

```
class Book:
    pass  # пустой класс, ничего не делает
```

Ключевое слово `class`, затем имя класса (принято с большой буквы, в стиле CamelCase), затем двоеточие и тело класса. Даже пустой класс должен иметь тело — мы используем `pass` как заглушку.

2.2. Создаём объект (экземпляр класса)

```
my_book = Book()    # Book() — вызов класса, создаёт новый объект
print(my_book)      # <__main__.Book object at 0x000002274AF46A50>
```

Что произошло? Python выделил память под новый объект типа `Book` и вернул ссылку на него. В выводе мы видим: модуль (`__main__`), имя класса и адрес в памяти (шестнадцатеричное число).

2.3. Добавляем атрибуты на лету

В Python объекты могут получать новые атрибуты даже после создания:

```
my_book.title = "Маленький принц"
my_book.author = "Антуан де Сент-Экзюпери"
print(my_book.title)  # Маленький принц
```

Но так делать не рекомендуется — это приводит к беспорядку. Вместо этого мы должны определить структуру объекта заранее, в методе `__init__`.

3. Конструктор `__init__` и параметр `self`

3.1. Что такое `__init__`?

`__init__` — это специальный метод, который автоматически вызывается сразу после создания объекта. Его задача — инициализировать объект начальными значениями.

```
1 class Book:
2     def __init__(self, title, author, pages):
3         self.title = title
4         self.author = author
5         self.pages = pages
6         self.in_stock = True  # значение по умолчанию
```

Разберём по частям:

- `def __init__(self, title, author, pages):` — метод принимает четыре параметра. Первый обязателен — `self`. Остальные мы передаём при создании.
- `self.title = title` — мы создаём у объекта (`self`) атрибут `title` и присваиваем ему значение параметра `title`.
- `self.in_stock = True` — атрибут со значением по умолчанию, его не нужно передавать.

3.2. Что такое `self`?

`self` — это ссылка на **текущий экземпляр** класса. Когда мы пишем `book = Book("Маленький принц", "Антуан де Сент-Экзюпери", 96)`, Python делает примерно так:

1. Создаёт пустой объект в памяти.
2. Вызывает метод `__init__`, передавая созданный объект как первый аргумент (`self`).
3. Внутри `__init__` мы через `self` добавляем атрибуты этому объекту.

Имя `self` не является ключевым словом — это просто соглашение. Можно использовать любое имя, но никогда так не делайте:

```
class BadExample:
    def __init__(myself, data):  # технически работает, но ужасно
        myself.data = data
```

3.3. Создание объектов с `__init__`

```
book1 = Book("Маленький принц", "Антуан де Сент-Экзюпери", 96)
book2 = Book("Приключения Тома Сойера", "Марк Твен", 240)

print(book1.title)   # Маленький принц
print(book2.author)  # Марк Твен
print(book1.in_stock)  # True
```

Каждый объект хранит свои собственные значения атрибутов. Они независимы.

4. Атрибуты класса и атрибуты экземпляра

4.1. Атрибуты класса

Атрибуты, определённые прямо в теле класса (не внутри методов), принадлежат самому классу и разделяются всеми экземплярами.

```
class Book:
    # атрибут класса
    type = "printed"

    def __init__(self, title):
        self.title = title   # атрибут экземпляра

print(Book.type)        # printed — доступ через класс
book = Book("Маленький принц")
print(book.type)        # printed — доступ через объект (идёт в класс)
```

Как это работает под капотом?
У каждого класса есть свой словарь `__dict__`, где хранятся атрибуты класса. У каждого объекта тоже есть свой `__dict__` с атрибутами экземпляра. Когда мы обращаемся к

`book.type`, Python сначала ищет `type` в словаре объекта, не находит — идёт в словарь класса.

Проверим:

```
print(Book.__dict__)  # {'__module__': '__main__', 'type': 'printed', ...}
print(book.__dict__)  # {'title': Маленький принц}
```

4.2. Когда использовать атрибуты класса?

- Для констант, общих для всех объектов.
- Для счётчиков созданных экземпляров.
- Для хранения данных, которые должны быть общими (но осторожно — изменение атрибута класса через один объект изменит его для всех).

Пример со счётчиком:

```
1   class Book:
2       total_books = 0
3
4       def __init__(self, title):
5           self.title = title
6           Book.total_books += 1  # обращаемся через класс, чтобы увеличить общий счётчик
7
8   b1 = Book("Маленький принц")
9   b2 = Book("Приключения Тома Сойера")
10  print(Book.total_books)  # 2
```

Если бы мы написали `self.total_books += 1`, то создали бы атрибут экземпляра с тем же именем, и счётчик бы не работал.

5. Методы экземпляра

Методы — это функции, которые принадлежат классу и работают с конкретным экземпляром.

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.in_stock = True

    def display(self):
        status = "в наличии" if self.in_stock else "нет"
        print(f"'{self.title}' {self.author} — {status}")

    def borrow(self):
        if self.in_stock:
            self.in_stock = False
            print(f"Вы взяли книгу '{self.title}'")
        else:
            print("Книги нет в наличии")
```

Важно: при вызове метода через объект, Python автоматически передаёт сам объект как первый аргумент (`self`).

```
book = Book("Маленький принц", " Антуан де Сент-Экзюпери")
book.display()   # Python преобразует в Book.display(book)
```

Методы могут принимать и другие параметры:

```
def renew(self, days):
    print(f"Книга '{self.title}' продлена на {days} дней")
```

6. Инкапсуляция: контроль доступа к данным

6.1. Публичные атрибуты (public)

По умолчанию все атрибуты в Python публичны — к ним можно обращаться напрямую.

```
book.in_stock = False   # работает
```

Но часто мы хотим защитить данные от некорректного изменения. Например, поле `in_stock` не должно становиться ничем, кроме True/False.

6.2. Защищённые атрибуты (protected) — одно подчёркивание `_`

Соглашение: если имя атрибута начинается с одного подчёркивания, разработчик предполагает, что это внутренний атрибут, и не следует обращаться к нему напрямую извне. Это только предупреждение, Python не блокирует доступ.

```
class Book:
```

```
    def __init__(self, title):
        self._title = title   # защищённый атрибут
```

Технически мы всё ещё можем написать `book._title`, но IDE могут подсвечивать это как предупреждение. Это этикет, а не защита.

6.3. Приватные атрибуты (private) — два подчёркивания `__`

Когда имя начинается с двух подчёркиваний, Python применяет **name mangling** (искажение имени). Атрибут переименовывается в `_ClassName__attribute`, что делает его труднодоступным случайно.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance   # приватный атрибут

    def get_balance(self):
        return self.__balance

acc = BankAccount("Иван", 1000)
print(acc.__balance)   # AttributeError: 'BankAccount' object has no attribute '__balance'
print(acc.get_balance())  # 1000
print(acc._BankAccount__balance)  # 1000 (но так делать не стоит!)
```

Зачем это нужно? Чтобы избежать конфликтов имён при наследовании и случайного доступа. Это не делает данные полностью приватными, но сигнализирует: «не трогай, это внутренняя кухня».

6.4. Свойства (property) — умный доступ

Часто мы хотим при обращении к атрибуту выполнить какую-то логику (проверку, вычисление). Для этого используются свойства.

Геттер и сеттер через property:

```python
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius   # защищённое поле

    @property
    def celsius(self):
        """Геттер — вызывается при обращении temp.celsius"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Сеттер — вызывается при присваивании temp.celsius = ..."""
        if value < -273.15:
            raise ValueError("Температура ниже абсолютного нуля")
        self._celsius = value

    @property
    def fahrenheit(self):
        """Свойство только для чтения (нет setter)"""
        return self._celsius * 9/5 + 32
```

Использование:

```python
t = Temperature(25)
print(t.celsius)      # 25 (вызвался геттер)
t.celsius = 30        # вызвался сеттер
# t.celsius = -300    # ValueError!
print(t.fahrenheit)   # 86.0 (вычисляемое свойство)
```

Обратите внимание: мы работаем с `t.celsius` так, будто это обычный атрибут, но на самом деле вызываются методы. Это мощный механизм для реализации инкапсуляции.

7. Статические методы и методы класса

7.1. Статические методы (`@staticmethod`)

Это методы, которые не получают ни `self`, ни `cls`. Они ведут себя как обычные функции, но лежат в пространстве имён класса. Удобны для вспомогательных операций, связанных с классом по смыслу.

```python
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def multiply(a, b):
        return a * b

print(MathUtils.add(5, 7))   # 12
```

Статические методы можно вызывать как через класс, так и через объект (но обычно через класс). Они не имеют доступа к атрибутам экземпляра или класса.

7.2. Методы класса (`@classmethod`)

Методы класса получают первым параметром сам класс (обычно называют `cls`), а не экземпляр. Они могут изменять атрибуты класса или создавать объекты класса (альтернативные конструкторы).

```python
class Person:
    species = "Homo sapiens"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species

    @classmethod
    def from_birth_year(cls, name, birth_year):
        """Альтернативный конструктор: создаёт Person по году рождения"""
        import datetime
        current_year = datetime.datetime.now().year
        age = current_year - birth_year
        return cls(name, age)   # Здесь cls — это Person, вызовет __init__
```

Использование:

```python
Person.change_species("Homme moderne")
print(Person.species)  # Homme moderne


p = Person.from_birth_year("Алиса", 1995)
print(p.age)           # например, 29
```

Зачем это нужно?

- `@classmethod` часто используется для создания объектов из данных в другом формате (например, из строки, из JSON).
- Они могут быть унаследованы и работать правильно в подклассах (в отличие от статических методов, которые жёстко привязаны к классу).

8. Итоговый пример: класс «Студент» с использованием всего изученного

```python
class Student:
    # Атрибут класса — счётчик созданных студентов
    total_students = 0

    def __init__(self, name, surname, student_id):
        self.name = name
        self.surname = surname
        self.__student_id = student_id   # приватный ID
        self._grades = []                # защищённый список оценок
        Student.total_students += 1

    # Публичный метод добавления оценки
    def add_grade(self, grade):
        if 2 <= grade <= 5:
            self._grades.append(grade)
        else:
            print("Оценка должна быть от 2 до 5")

    # Свойство для среднего балла (только чтение)
    @property
    def average_grade(self):
        if not self._grades:
            return 0.0
        return sum(self._grades) / len(self._grades)

    # Свойство для чтения ID (геттер)
    @property
    def student_id(self):
        return self.__student_id

```

```python
    # Метод класса: создать студента из строки "Имя Фамилия ID"
    @classmethod
    def from_string(cls, data):
        name, surname, sid = data.split()
        return cls(name, surname, int(sid))

    # Статический метод: проверка корректности ID
    @staticmethod
    def is_valid_id(sid):
        return 10000 <= sid <= 99999

# Примеры использования
s1 = Student("Иван", "Петров", 12345)
s2 = Student.from_string("Мария Иванова 54321")

s1.add_grade(5)
s1.add_grade(4)
s1.add_grade(3)

print(f"Средний балл: {s1.average_grade:.2f}")  # Средний балл: 4.00
print(f"Всего студентов: {Student.total_students}")  # 2

# Проверка ID
print(Student.is_valid_id(12345))        # True
print(Student.is_valid_id(12))           # False

# Попытка напрямую изменить приватный ID:
# s1.__student_id = 99999  # AttributeError
# Но можно обойти, если очень захотеть:
# s1._Student__student_id = 99999  # работает, но мы предупреждены
```